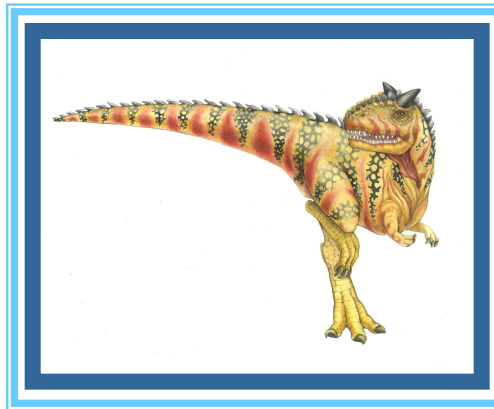


Chapter 7: Deadlocks





Chapter 7: Deadlocks

- The Deadlock Problem
- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock





Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks
- To present a number of different methods for preventing or avoiding deadlocks in a computer system









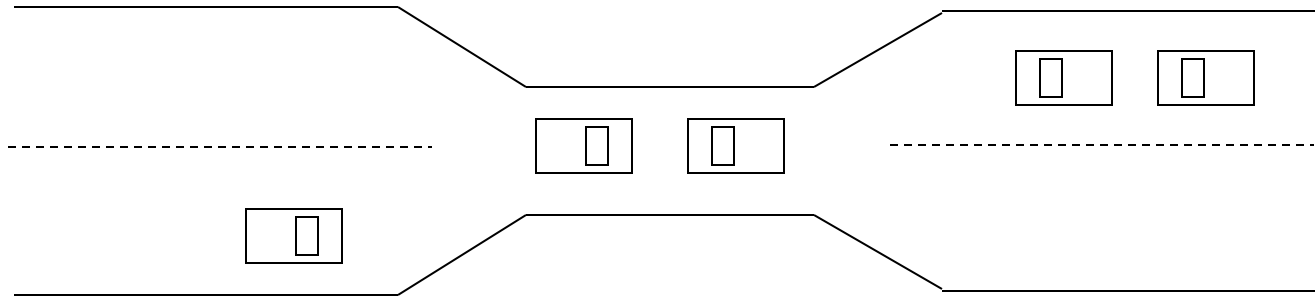
The Deadlock Problem

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set
- Example
 - System has 2 disk drives
 - P_1 and P_2 each hold one disk drive and each needs another one
- Example
 - semaphores A and B , initialized to 1 P_0 P_1
wait (A); wait(B) wait (B); wait(A)





Bridge Crossing Example



- Traffic only in one direction
- Each section of a bridge can be viewed as a resource
- If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback)
- Several cars may have to be backed up if a deadlock occurs
- Starvation is possible
- Note – Most OSes do not prevent or deal with deadlocks





System Model

- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**











Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .





Resource-Allocation Graph

A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$





Resource-Allocation Graph (Cont.)

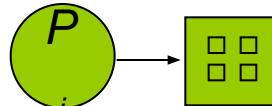
- Process



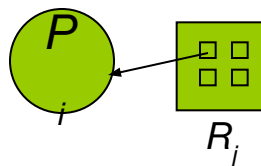
- Resource Type with 4 instances



- P_i requests instance of R_j

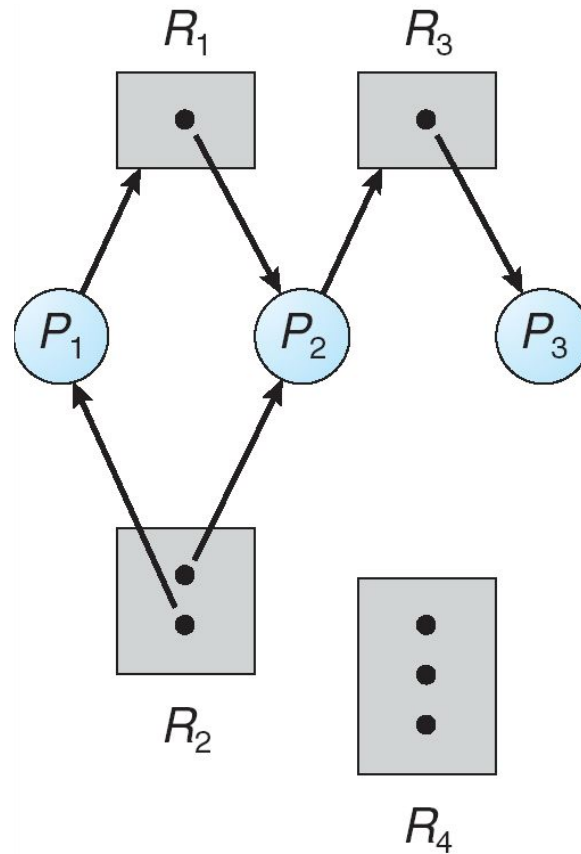


- P_i is holding an instance of R_j



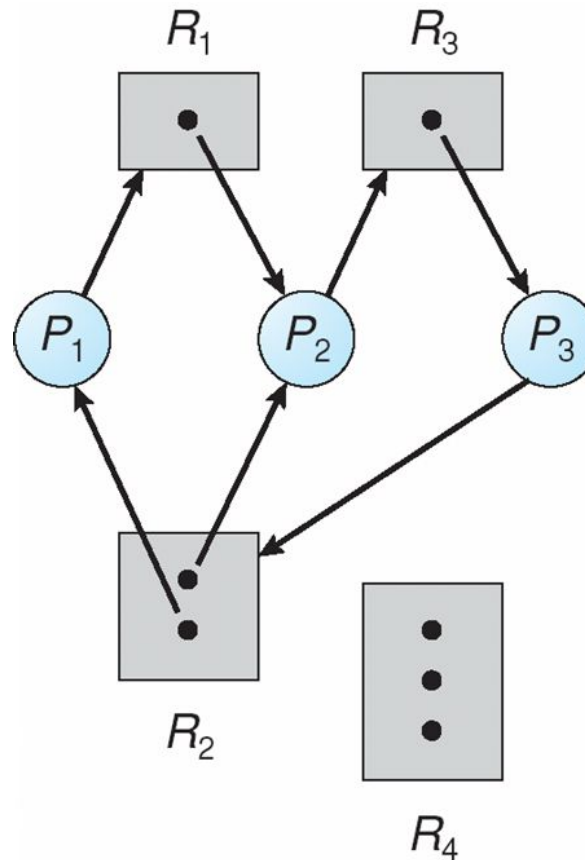


Example of a Resource Allocation Graph



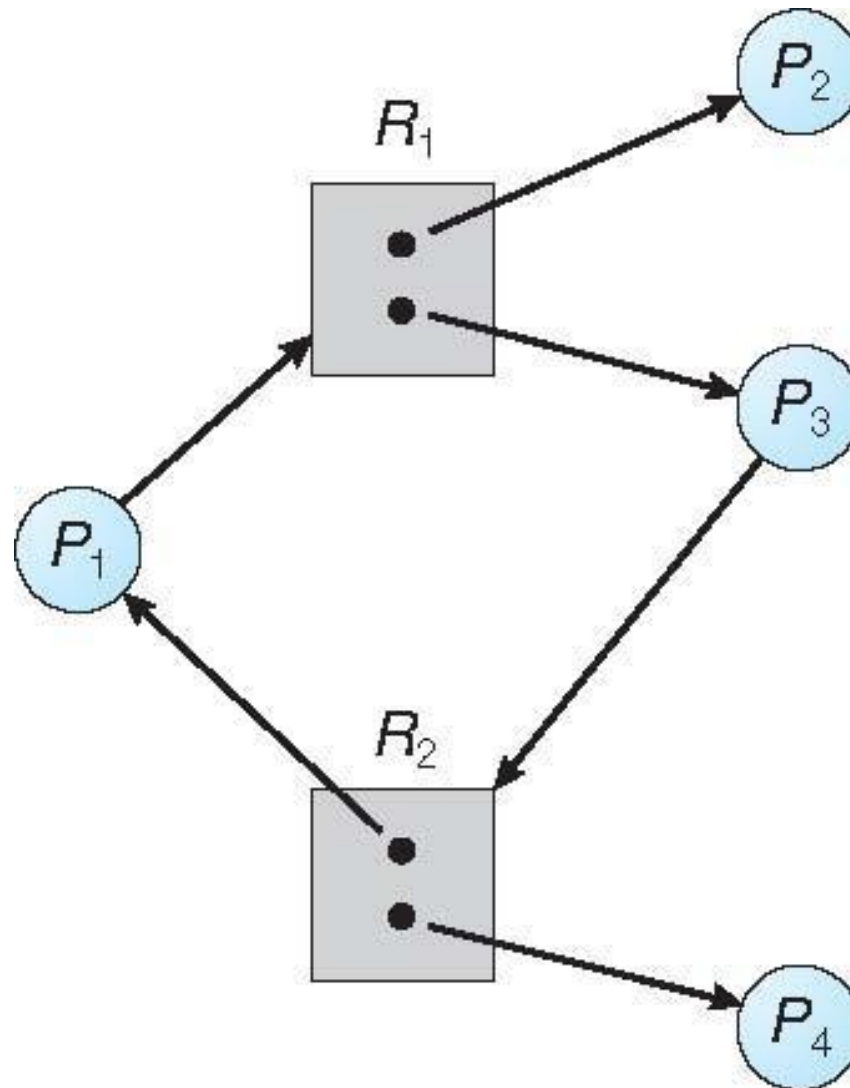


Resource Allocation Graph With A Deadlock





Graph With A Cycle But No Deadlock





Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock





Methods for Handling Deadlocks

- Ensure that the system will **never** enter a deadlock state
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX





Deadlock Prevention

Restrain the ways request can be made

- **Mutual Exclusion** – not required for sharable resources; must hold for nonsharable resources
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none
 - DVD File on disk printer example
 - Low resource utilization; starvation possible





Deadlock Prevention (Cont.)

- **No Preemption** –
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration





Deadlock Avoidance

Requires that the system has some additional *a priori* information available

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes





Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$
- That is:
 - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on





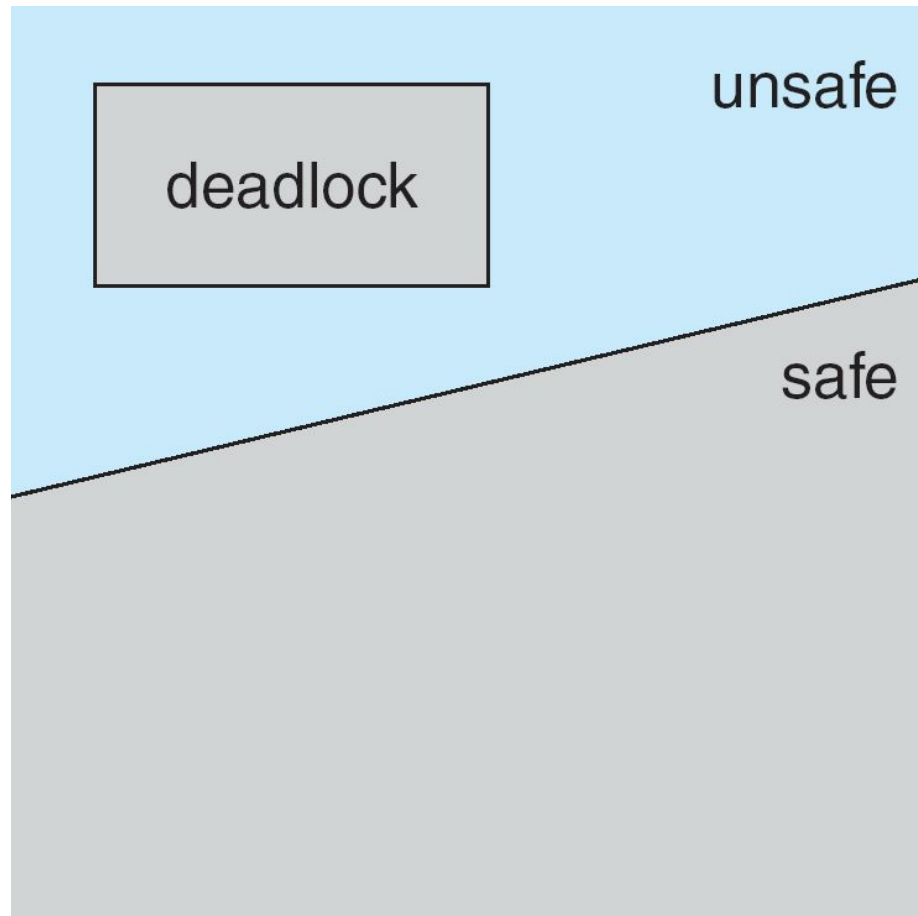
Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.





Safe, Unsafe, Deadlock State





Avoidance algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm





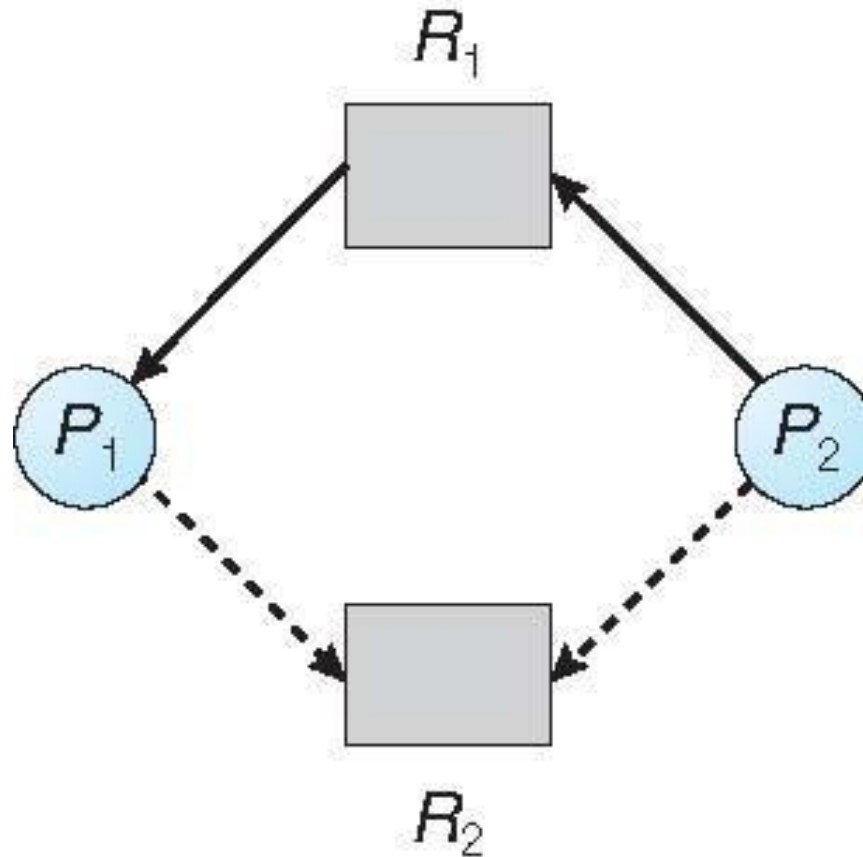
Resource-Allocation Graph Scheme

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system



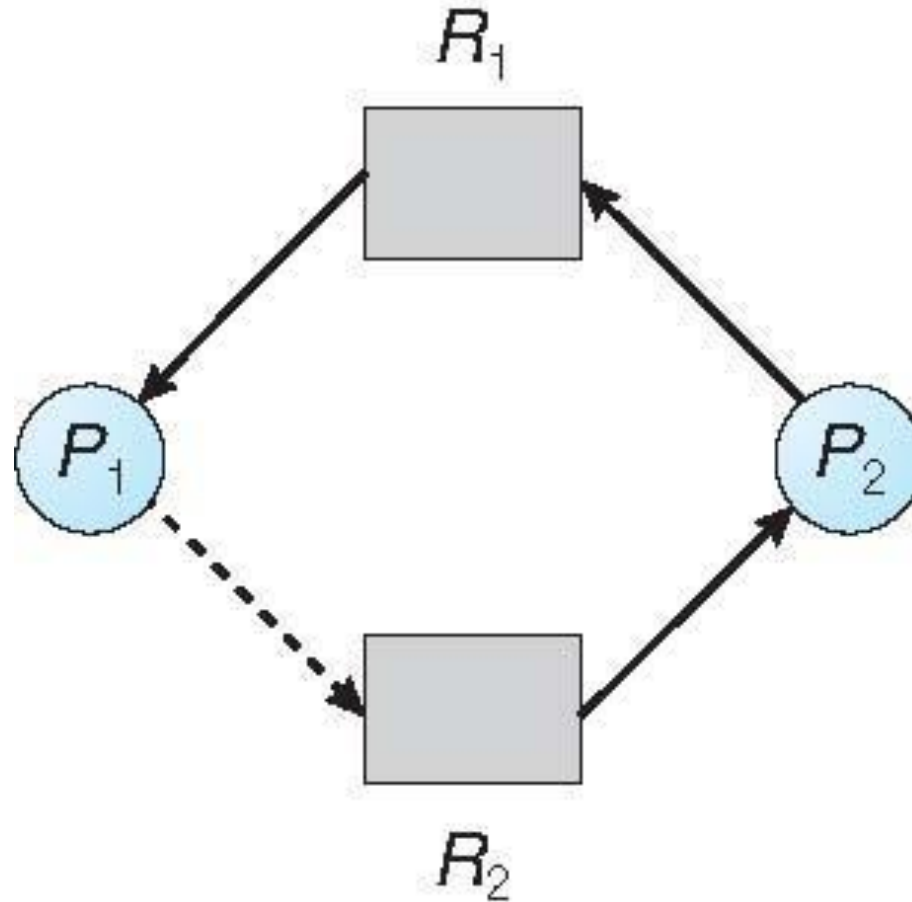


Resource-Allocation Graph





Unsafe State In Resource-Allocation Graph





Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph





Banker's Algorithm

- Multiple instances
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time













Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$





Safety Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively.
Initialize:

Work = *Available*

Finish [*i*] = *false* for *i* = 0, 1, ..., *n*- 1

2. Find an *i* such that both:

(a) *Finish* [*i*] = *false*

(b) $Need_i \leq Work$

If no such *i* exists, go to step 4

3. *Work* = *Work* + *Allocation*_{*i*}
Finish[*i*] = *true*
go to step 2

4. If *Finish* [*i*] == *true* for all *i*, then the system is in a safe state





Resource-Request Algorithm for Process P_i

$Request$ = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A\ B\ C$	$A\ B\ C$	$A\ B\ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	





Example (Cont.)

- The content of the matrix *Need* is defined to be *Max – Allocation*

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria





Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?





Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme





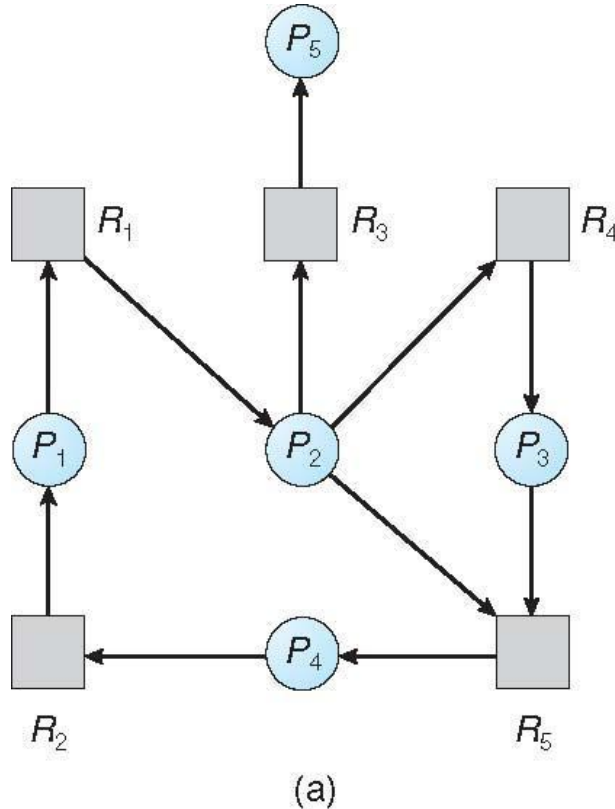
Single Instance of Each Resource Type

- Maintain *wait-for* graph
 - Nodes are processes
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

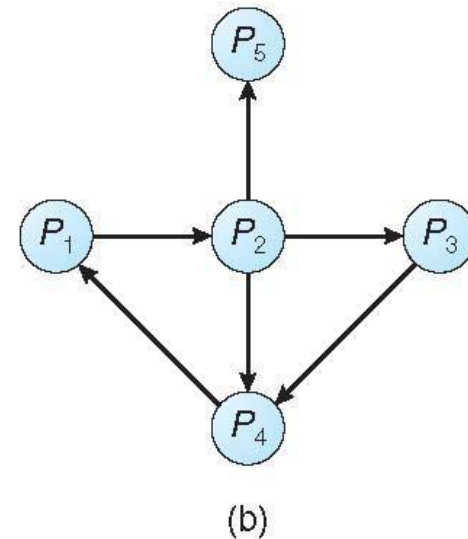




Resource-Allocation Graph and Wait-for Graph



Resource-Allocation
Graph



Corresponding wait-for
graph





Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .





Detection Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively
Initialize:

(a) *Work* = *Available*

(b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
 $Finish[i] = \text{false}$; otherwise, $Finish[i] = \text{true}$

2. Find an index i such that both:

(a) $Finish[i] == \text{false}$

(b) $Request_i \leq Work$

If no such i exists, go to step 4





Detection Algorithm (Cont.)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2
4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state





Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0		0	1	0			0	0	0
P_1		2	0	0			2	0	2
P_2			3	0	3		0	0	0
P_3	2	1	1		1	0	0		
P_4	0	0	2		0	0	2		

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i





Example (Cont.)

- P_2 requests an additional instance of type C

Request

A B C

P_0 0 0 0

P_1 2 0 2

P_2 0 0 1

P_3 1 0 0

P_4 0 0 2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4





Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - 4 one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.





Recovery from Deadlock: Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
 - Priority of the process
 - How long process has computed, and how much longer to completion
 - Resources the process has used
 - Resources process needs to complete
 - How many processes will need to be terminated
 - Is process interactive or batch?





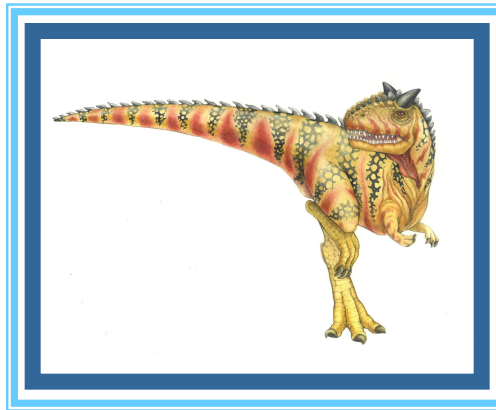
Recovery from Deadlock: Resource Preemption

- Selecting a victim – minimize cost
- Rollback – return to some safe state, restart process for that state
- Starvation – same process may always be picked as victim, include number of rollback in cost factor

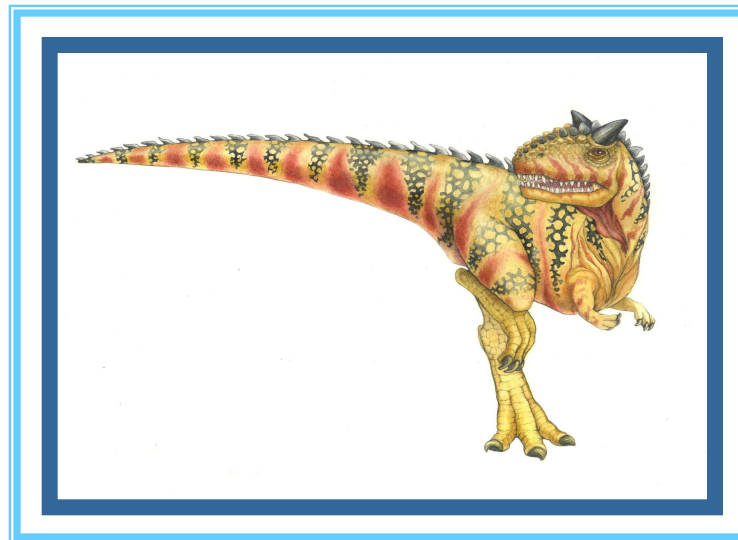




End of Chapter 7



Process Synchronization





Background

- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **count** that keeps track of the number of full buffers. Initially, count is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {  
  
    /* produce an item and put in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; // do nothing  
    buffer [in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
  
}
```

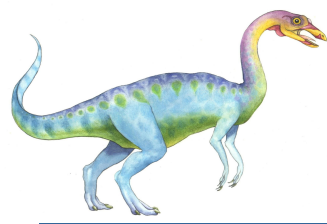




Consumer

```
while (true) {  
    while (counter == 0)  
        ; // do nothing  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
  
    /* consume the item in nextConsumed */  
}
```





Race Condition

- `counter++` could be implemented as

A: `register1 = counter`

B: `register1 = register1 + 1`

C: `counter = register1`

- `counter--` could be implemented as

D: `register2 = counter`

E: `register2 = register2 - 1`

F: `count = register2`

- Consider this execution interleaving with “count = 5” initially:

A: producer execute `register1 = counter` {`register1 = 5`}

B: producer execute `register1 = register1 + 1` {`register1 = 6`}

D: consumer execute `register2 = counter` {`register2 = 5`}

E: consumer execute `register2 = register2 - 1` {`register2 = 4`}

C: producer execute `counter = register1` {`count = 6`}

F: consumer execute `count = register2` {`count = 4`}





Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- Critical section problem is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- Especially challenging with preemptive kernels





Critical Section

- General structure of process p_i is

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

Figure 6.1 General structure of a typical process P_i .

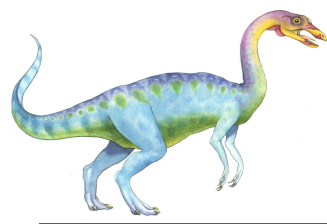




Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes





Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
 - int **turn**;
 - Boolean **flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i]** = true implies that process **P_i** is ready!





Algorithm for Process P_i

```
do {  
    flag[i] = TRUE;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = FALSE;  
    remainder section  
} while (TRUE);
```

- Provable that
 1. Mutual exclusion is preserved
 2. Progress requirement is satisfied
 3. Bounded-waiting requirement is met





Synchronization Hardware

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - 4 Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - 4 **Atomic = non-interruptable**
 - Either test memory word and set value
 - Or swap contents of two memory words





Solution to Critical-section Problem Using Locks

do {

 acquire lock

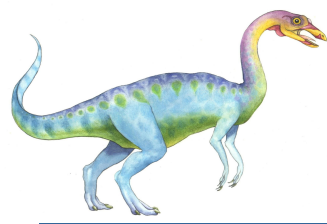
 critical section

 release lock

 remainder section

} while (TRUE);





TestAndSet Instruction

- Definition:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```



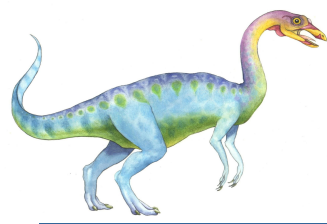


Solution using TestAndSet

- Shared boolean variable lock, initialized to FALSE
- Solution:

```
do {  
    while ( TestAndSet (&lock ))  
        ; // do nothing  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
  
} while (TRUE);
```



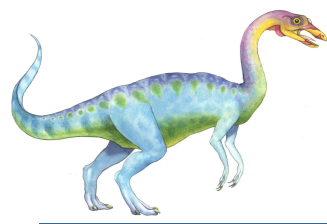


Swap Instruction

- Definition:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```





Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key
- Solution:

```
do {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
  
        // critical section  
  
    lock = FALSE;  
  
        // remainder section  
  
} while (TRUE);
```





Bounded-waiting Mutual Exclusion with TestAndSet()

```
do {  
    waiting[i] = TRUE;  
    key = TRUE;  
    while (waiting[i] && key)  
        key = TestAndSet(&lock);  
    waiting[i] = FALSE;  
    // critical section  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
    if (j == i)  
        lock = FALSE;  
    else  
        waiting[j] = FALSE;  
    // remainder section  
} while (TRUE);
```





Semaphore

- Synchronization tool that does not require busy waiting
- Semaphore S – integer variable
- Two standard operations modify S : `wait()` and `signal()`
 - Originally called `P()` and `V()`
- Less complicated
- Can only be accessed via two indivisible (atomic) operations
 - `wait (S) {`
 - `while S <= 0`
 - `; // no-op`
 - `S--;`
 - `}`
 - `signal (S) {`
 - `S++;`
 - `}`





Semaphore as General Synchronization Tool

- **Counting** semaphore – integer value can range over an unrestricted domain
- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
 - Also known as **mutex locks**
- Can implement a counting semaphore **S** as a binary semaphore
- Provides mutual exclusion

```
Semaphore mutex; // initialized to 1
do {
    wait (mutex);
    // Critical Section
    signal (mutex);
    // remainder section
} while (TRUE);
```





Semaphore Implementation

- Must guarantee that no two processes can execute `wait ()` and `signal ()` on the same semaphore at the same time
- Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - 4 But implementation code is short
 - 4 Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution





Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

```
typedef struct  
{  
    int value;  
    struct process *list;  
} semaphore;
```





Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

- Implementation of signal:

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```





Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0
wait (S);
wait (Q);

•
•
•

signal (S);
signal (Q);

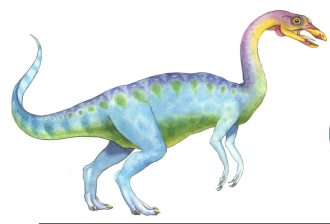
P_1

wait (Q);
wait (S);

signal (Q);
signal (S);

- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
 - Solved via **priority-inheritance protocol**





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem





Bounded-Buffer Problem

- N buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N





Bounded Buffer Problem (Cont.)

- The structure of the producer process

do {

 // produce an item in nextp

 wait (empty);

 wait (mutex);

 // add the item to the buffer

 signal (mutex);

 signal (full);

} while (TRUE);





Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
do {  
    wait (full);  
    wait (mutex);  
  
    // remove an item from buffer to nextc  
  
    signal (mutex);  
    signal (empty);  
  
    // consume the item in nextc  
  
} while (TRUE);
```

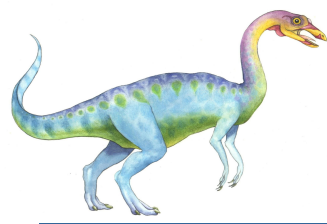




Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are treated – all involve priorities
- Shared Data
 - Data set
 - Semaphore **mutex** initialized to 1
 - Semaphore **wrt** initialized to 1
 - Integer **readcount** initialized to 0





Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {  
    wait (wrt) ;  
  
    //  writing is performed  
  
    signal (wrt) ;  
} while (TRUE);
```





Readers-Writers Problem (Cont.)

- The structure of a reader process

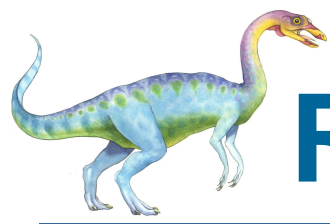
do {

```
    wait (mutex) ;  
    readcount ++ ;  
    if (readcount == 1)  
        wait (wrt) ;  
    signal (mutex)
```

// reading is performed

```
    wait (mutex) ;  
    readcount - - ;  
    if (readcount == 0)  
        signal (wrt) ;  
    signal (mutex) ;  
} while (TRUE);
```





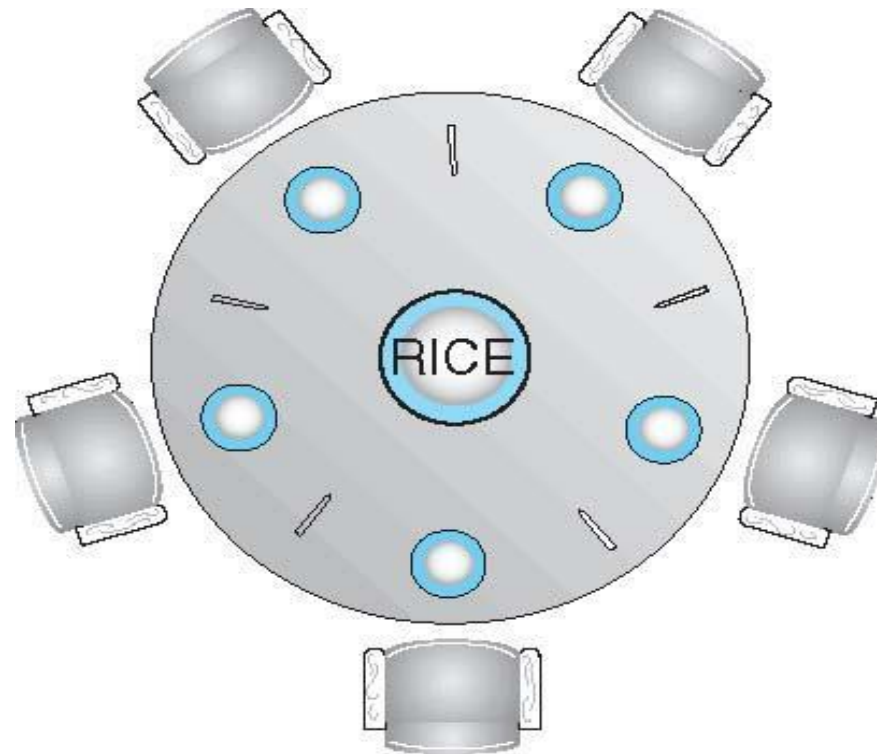
Readers-Writers Problem Variations

- *First* variation – no reader kept waiting unless writer has permission to use shared object
- *Second* variation – once writer is ready, it performs write asap
- Both may have starvation leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks





Dining-Philosophers Problem



- Philosophers spend their lives thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers
 - Shared data
 - 4 Bowl of rice (data set)
 - 4 Semaphore **chopstick** [5] initialized to 1





Dining-Philosophers Problem Algorithm

- The structure of Philosopher i :

```
do {  
    wait ( chopstick[i] );  
    wait ( chopStick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- What is the problem with this algorithm?



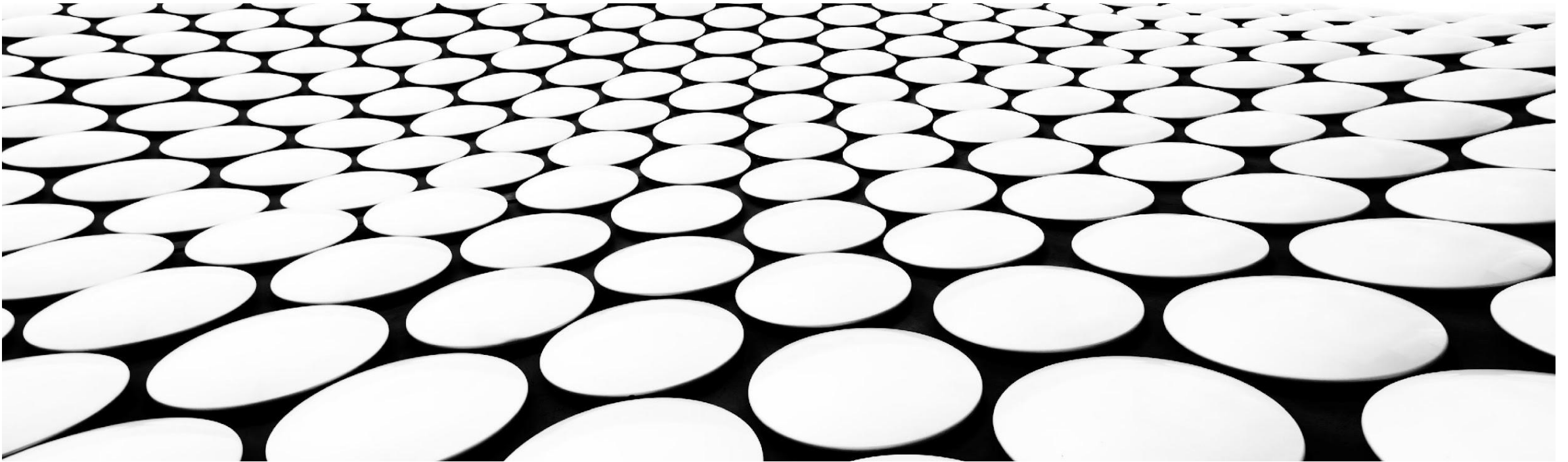


Problems with Semaphores

- Incorrect use of semaphore operations:
 - signal (mutex) wait (mutex)
 - wait (mutex) ... wait (mutex)
 - Omitting of wait (mutex) or signal (mutex) (or both)
- Deadlock and starvation

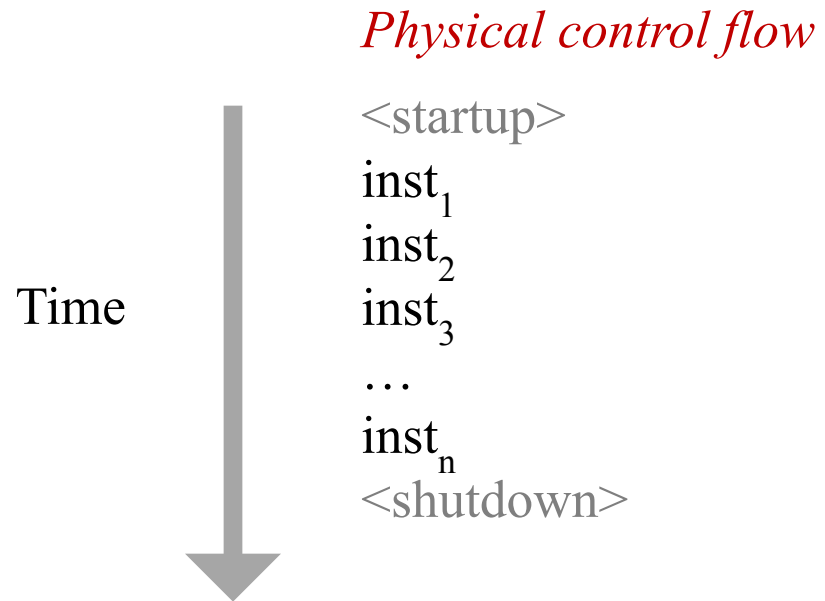


EXCEPTIONS



CONTROL FLOW

- Processors do only one thing:
 - From startup to shutdown, a CPU simply reads and executes (interprets) a sequence of instructions, one at a time
 - This sequence is the CPU's *control flow* (or *flow of control*)



ALTERING THE CONTROL FLOW

- Traditionally, two mechanisms for changing control flow:
 - Jumps and branches
 - Call and return

React to changes in *program state*

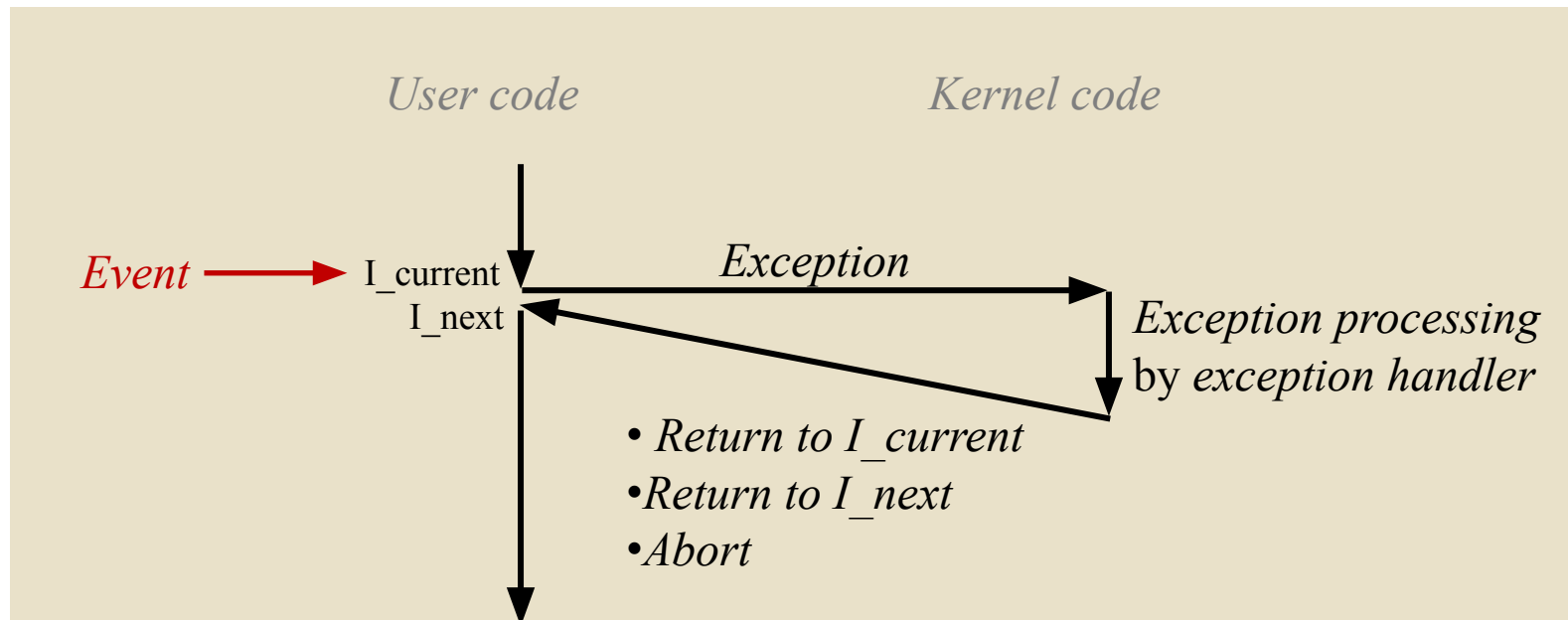
- Insufficient for a useful system:
Difficult to react to changes in *system state*
 - Data arrives from a disk or a network adapter
 - Instruction divides by zero
 - User hits Ctrl-C at the keyboard
 - System timer expires
- System needs mechanisms for “exceptional control flow”

EXCEPTIONAL CONTROL FLOW

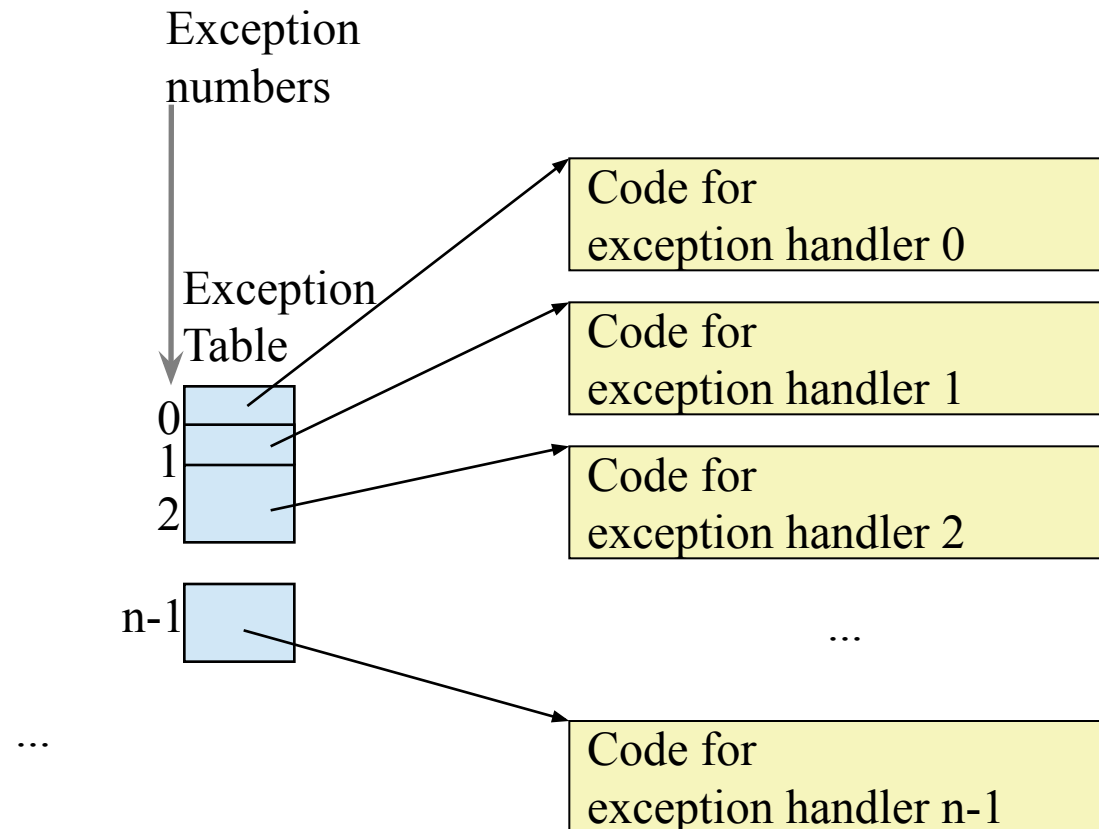
- Exists at all levels of a computer system
- Low level mechanisms
 - **1. Exceptions**
 - Change in control flow in response to a system event (i.e., change in system state)
 - Implemented using combination of hardware and OS software
- Higher level mechanisms
 - **2. Process context switch**
 - Implemented by OS software and hardware timer
 - **3. Signals**
 - Implemented by OS software
 - **4. Nonlocal jumps: setjmp() and longjmp()**
 - Implemented by C runtime library

EXCEPTIONS

- An *exception* is an abrupt change in the control flow in response to some changes in the processor's state (a transfer of control to the OS *kernel* in response to some *event*)
- Examples of events: Divide by 0, arithmetic overflow, page fault, I/O request completes, typing Ctrl-C



EXCEPTION TABLES



- Each type of event has a unique exception number k
- k = index into exception table (a.k.a. interrupt vector)
- Handler k is called each time exception k occurs

ASYNCHRONOUS EXCEPTIONS (INTERRUPTS)

- Caused by events external to the processor –Network adapters, disk controllers, timer chips
 - Indicated by setting the processor's *interrupt pin*
 - Handler returns to “next” instruction
- Examples:
 - Timer interrupt
 - Every few ms, an external timer chip triggers an interrupt
 - Used by the kernel to take back control from user programs
 - I/O interrupt from external device
 - Hitting Ctrl-C at the keyboard
 - Arrival of a packet from a network
 - Arrival of data from a disk

SYNCHRONOUS EXCEPTIONS

- Caused by events that occur as a result of executing an instruction:

- ***Traps***

- Intentional- user program request services
- Examples: ***system calls***, breakpoint traps, special instructions- fork(), read(), execve()
- Returns control to “next” instruction

- ***Faults***

- Unintentional but possibly recoverable
- Examples: page faults (recoverable), protection faults (unrecoverable), floating point exceptions
- Either re-executes faulting (“current”) instruction or aborts

- ***Aborts***

- Unintentional and unrecoverable
- Examples: illegal instruction, parity error, machine check
- Aborts current program

EXCEPTIONS IN LINUX/X86-64 SYSTEMS

- Linux/x86-64: Faults and Aborts
 - Divide Error (exception 0): An application attempt to divide by zero or the result of a divide instruction is too big for the destination operand- floating exception
 - General Protection Fault (exception 13): A program references an undefined area of virtual memory or the program attempt to write to a read-only text segment- segmentation Fault
 - Page Fault (exception 14)
 - Machine Check (exception 18): Fatal hardware error

SYSTEM CALLS

- Each x86-64 system call has a unique ID number correspond to an offset in a jump table in the kernel
- Examples:

<i>Number</i>	<i>Name</i>	<i>Description</i>
0	read	Read file
1	write	Write file
2	open	Open file
3	close	Close file
4	stat	Get info about file
57	fork	Create process
59	execve	Execute a program
60	_exit	Terminate process
62	kill	Send signal to process

EXCEPTIONS IN LINUX/X86-64 SYSTEMS

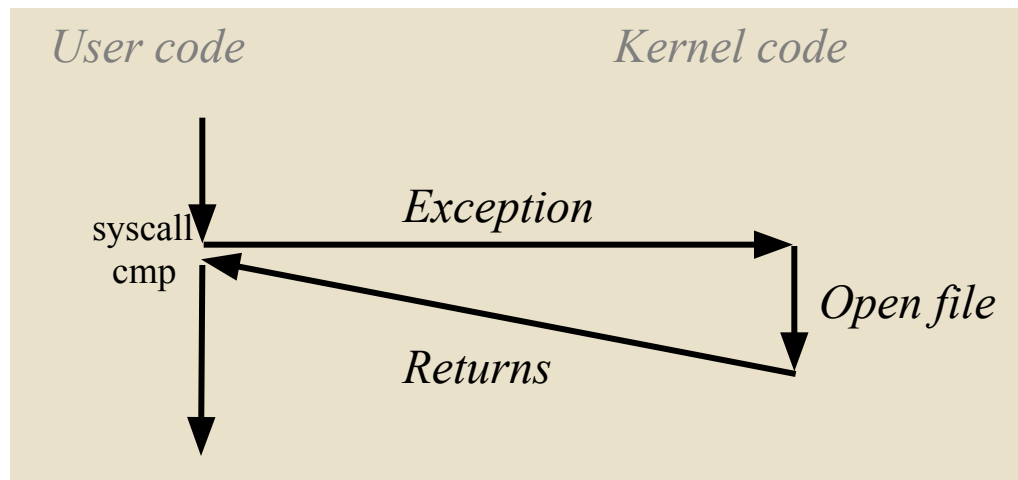
■ Linux/x86-64: System call

Number	Name	Description	Number	Name	Description
0	read	Read file	33	pause	Suspend process until signal arrives
1	write	Write file	37	alarm	Schedule delivery of alarm signal
2	open	Open file	39	getpid	Get process ID
3	close	Close file	57	fork	Create process
4	stat	Get info about file	59	execve	Execute a program
9	mmap	Map memory page to file	60	_exit	Terminate process
12	brk	Reset the top of the heap	61	wait4	Wait for a process to terminate
32	dup2	Copy file descriptor	62	kill	Send signal to a process

SYSTEM CALL EXAMPLE: OPENING FILE

- User calls: `open(filename, options)`
- Calls `__open` function, which invokes system call instruction `syscall`

```
0000000000e5d70 <__open>:  
...  
e5d79: b8 02 00 00 00    mov $0x2,%eax # open is syscall #2  
e5d7e: 0f 05             syscall      # Return value in %rax  
e5d80: 48 3d 01 f0 ff ff  cmp $0xffffffff001,%rax  
...  
e5dfa: c3               retq
```



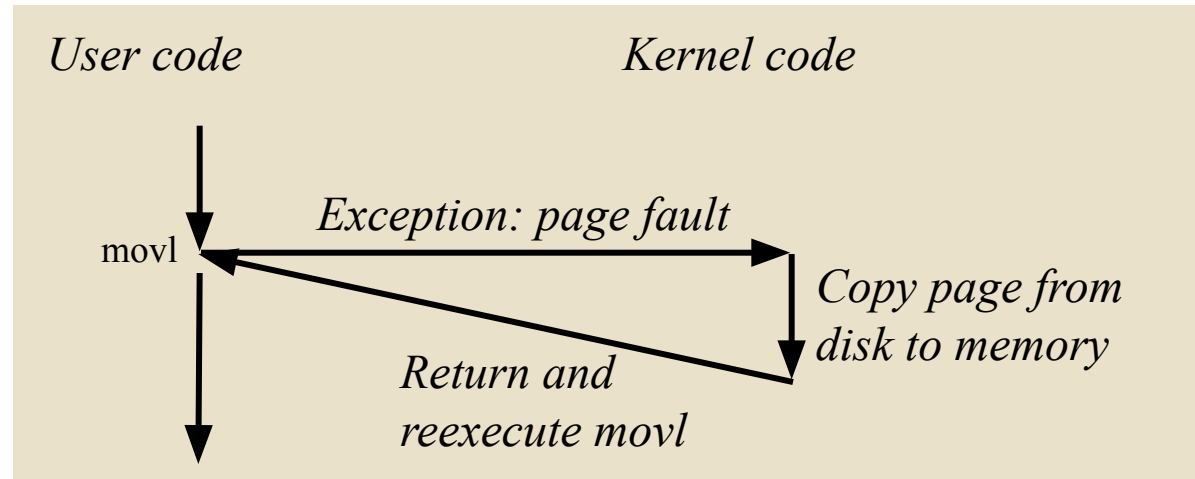
- `%rax` contains syscall number
- Other arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`
- Return value in `%rax`
- Negative value is an error corresponding to negative `errno`

FAULT EXAMPLE: PAGE FAULT

- User writes to memory location
- That portion (page) of user's memory is currently on disk

```
int a[1000];  
main ()  
{  
    a[500] = 13;  
}
```

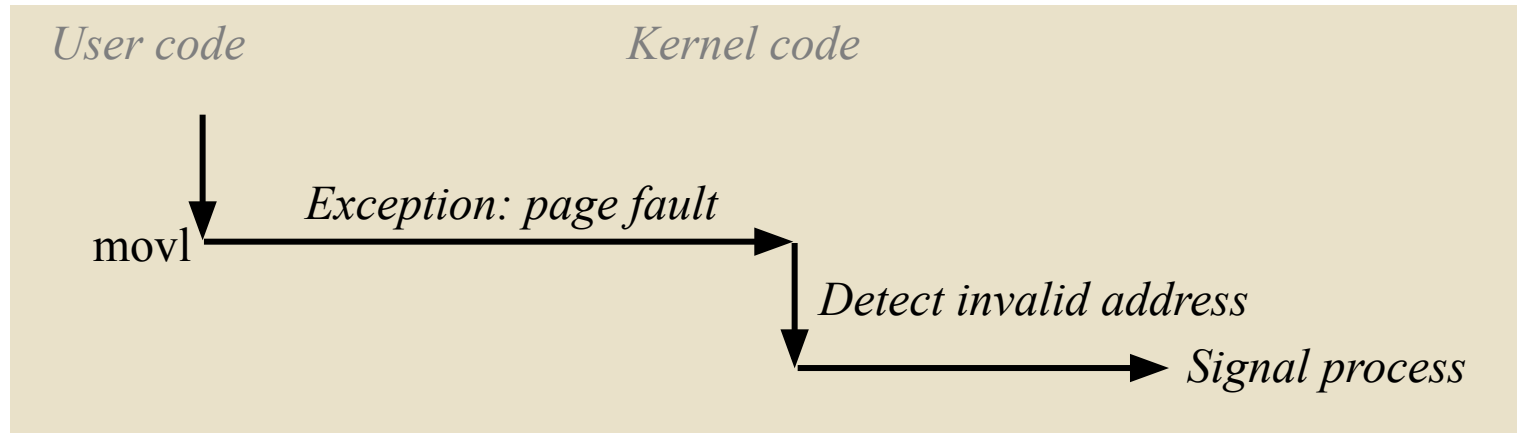
```
80483b7: c7 05 10 9d 04 08 0d movl $0xd,0x8049d10
```



FAULT EXAMPLE: INVALID MEMORY REFERENCE

```
int a[1000];  
main ()  
{  
    a[5000] = 13;  
}
```

80483b7: c7 05 60 e3 04 08 0d movl \$0xd,0x804e360



- Sends SIGSEGV signal to user process
- User process exits with “**segmentation fault**”

SIGNALS

- A *signal* is a small message that notifies a process that an event of some type has occurred in the system
- Low level hardware exceptions are processed by the kernels exception handlers and not visible to user process
- Signals provide a mechanism for exposing the occurrence of such exception to user processes
 - Akin to exceptions and interrupts
 - Sent from the kernel (sometimes at the request of another process) to a process
 - Signal type is identified by small integer ID's (1-30)
 - Only information in a signal is its ID and the fact that it arrived

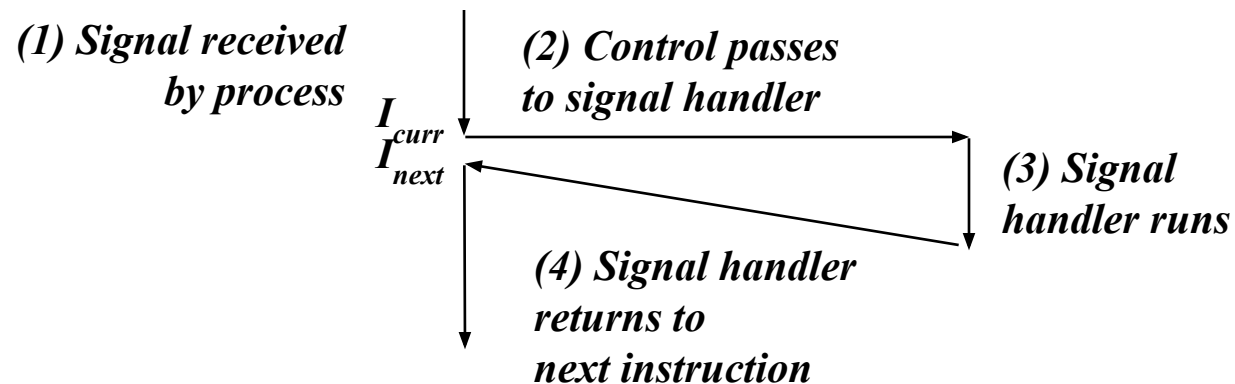
<i>ID</i>	<i>Name</i>	<i>Default Action</i>	<i>Corresponding Event</i>
2	SIGINT	Terminate	User typed ctrl-c
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

SIGNAL CONCEPTS: SENDING A SIGNAL

- Kernel *sends* (delivers) a signal to a *destination process* by updating some state in the context of the destination process
- Kernel sends a signal for one of the following reasons:
 - Kernel has detected a system event such as divide-by-zero (SIGFPE) or the termination of a child process (SIGCHLD)
 - Another process has invoked the **kill** system call to explicitly request the kernel to send a signal to the destination process

SIGNAL CONCEPTS: RECEIVING A SIGNAL

- A destination process *receives* a signal when it is forced by the kernel to react in some way to the delivery of the signal
- Some possible ways to react:
 - *Ignore* the signal (do nothing)
 - *Terminate* the process (with optional core dump)
 - *Catch* the signal by executing a user-level function called *signal handler*
 - Akin to a hardware exception handler being called in response to an asynchronous interrupt:



SIGNAL CONCEPTS: PENDING AND BLOCKED SIGNALS

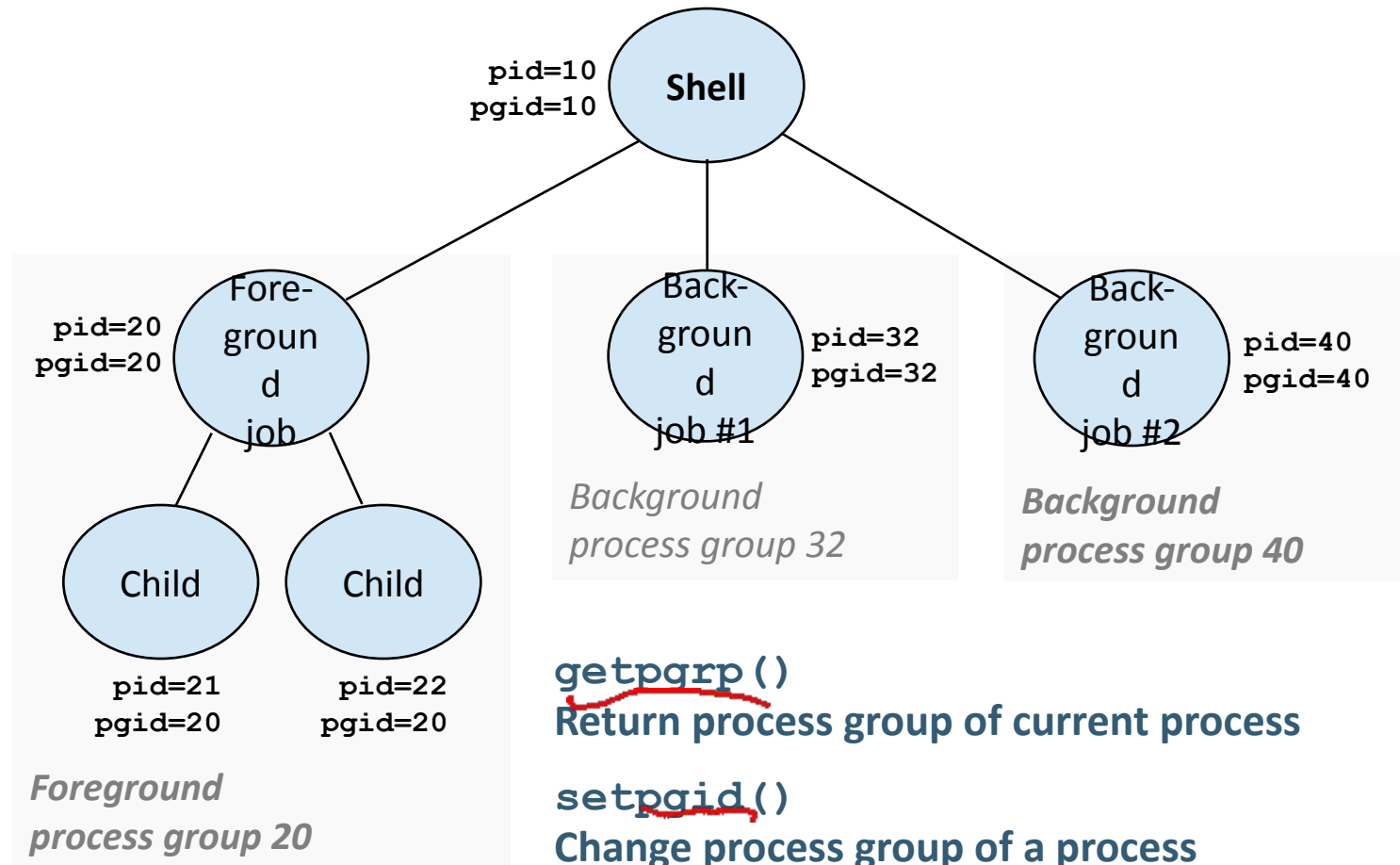
- A signal is *pending* if sent but not yet received
 - There can be at most one pending signal of any particular type
 - Important: Signals are not queued
 - If a process has a pending signal of type k, then subsequent signals of type k that are sent to that process are discarded
- A process can *block* the receipt of certain signals
 - Blocked signals can be delivered, but will not be received until the signal is unblocked
- A pending signal is received at most once

SIGNAL CONCEPTS: PENDING/BLOCKED BITS

- Kernel maintains pending and blocked bit vectors in the context of each process
 - **pending**: represents the set of pending signals
 - Kernel sets bit k in **pending** when a signal of type k is delivered
 - Kernel clears bit k in **pending** when a signal of type k is received
 - **blocked**: represents the set of blocked signals
 - Can be set and cleared by using the **sigprocmask** function
 - Also referred to as the *signal mask*.

SENDING SIGNALS: PROCESS GROUPS

- Every process belongs to exactly one process group



SENDING SIGNALS WITH /BIN/KILL PROGRAM

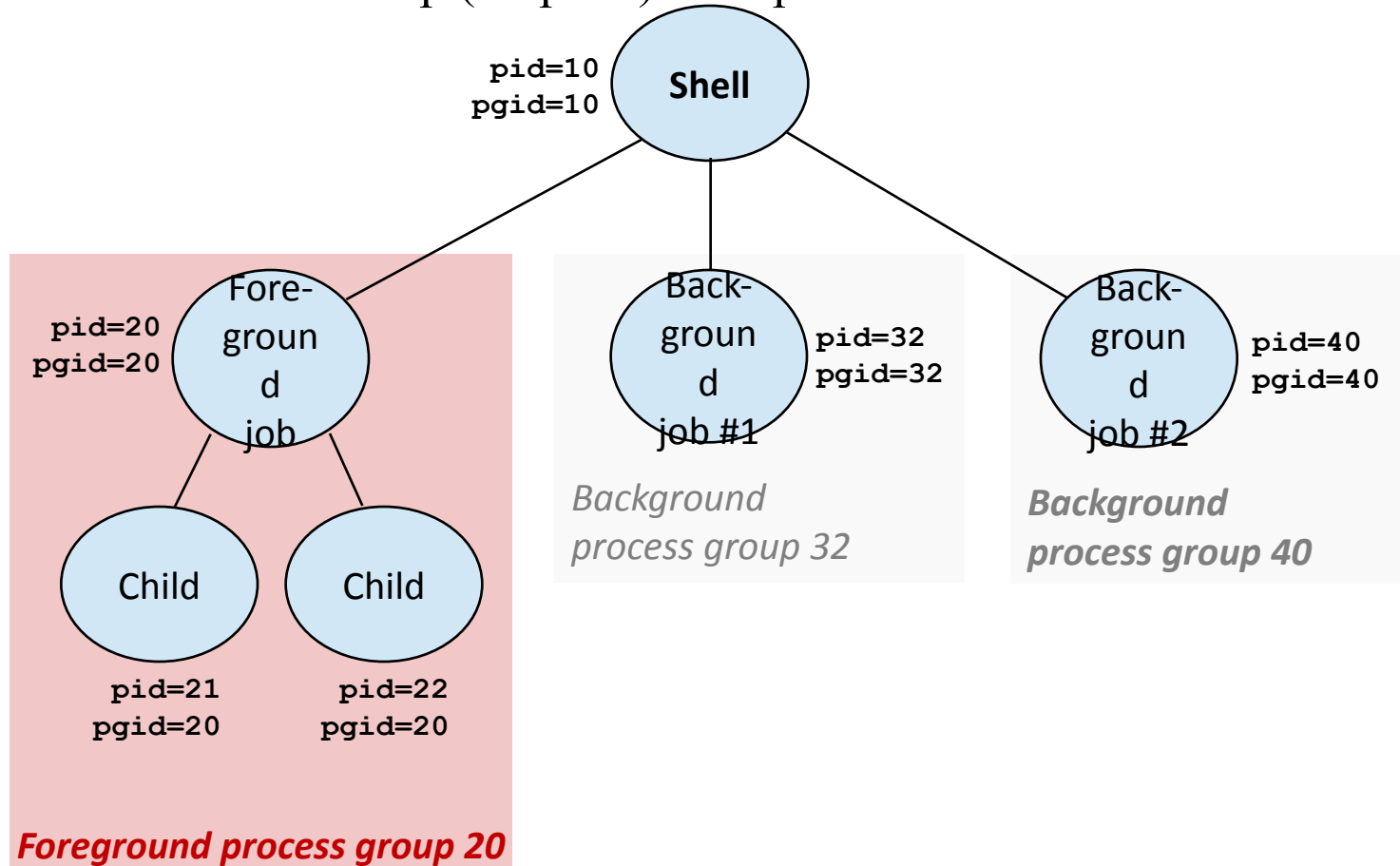
- /bin/kill program sends arbitrary signal to a process or process group
- Examples
- **/bin/kill -9 24818**
Send SIGKILL to process 24818
- **/bin/kill -9 -24817**
Send SIGKILL to every process in process group 24817

```
linux> ./forks 16
Child1: pid=24818 pgrp=24817
Child2: pid=24819 pgrp=24817

linux> ps
  PID TTY          TIME CMD
 24788 pts/2        00:00:00 tcsh
 24818 pts/2        00:00:02 forks
 24819 pts/2        00:00:02 forks
 24820 pts/2        00:00:00 ps
linux> /bin/kill -9 -24817
linux> ps
  PID TTY          TIME CMD
 24788 pts/2        00:00:00 tcsh
 24823 pts/2        00:00:00 ps
linux>
```

SENDING SIGNALS FROM THE KEYBOARD

- Typing ctrl-c (ctrl-z) causes the kernel to send a SIGINT (SIGTSTP) to every job in the foreground process group.
- SIGINT – default action is to terminate each process
- SIGTSTP – default action is to stop (suspend) each process



RECEIVING SIGNALS

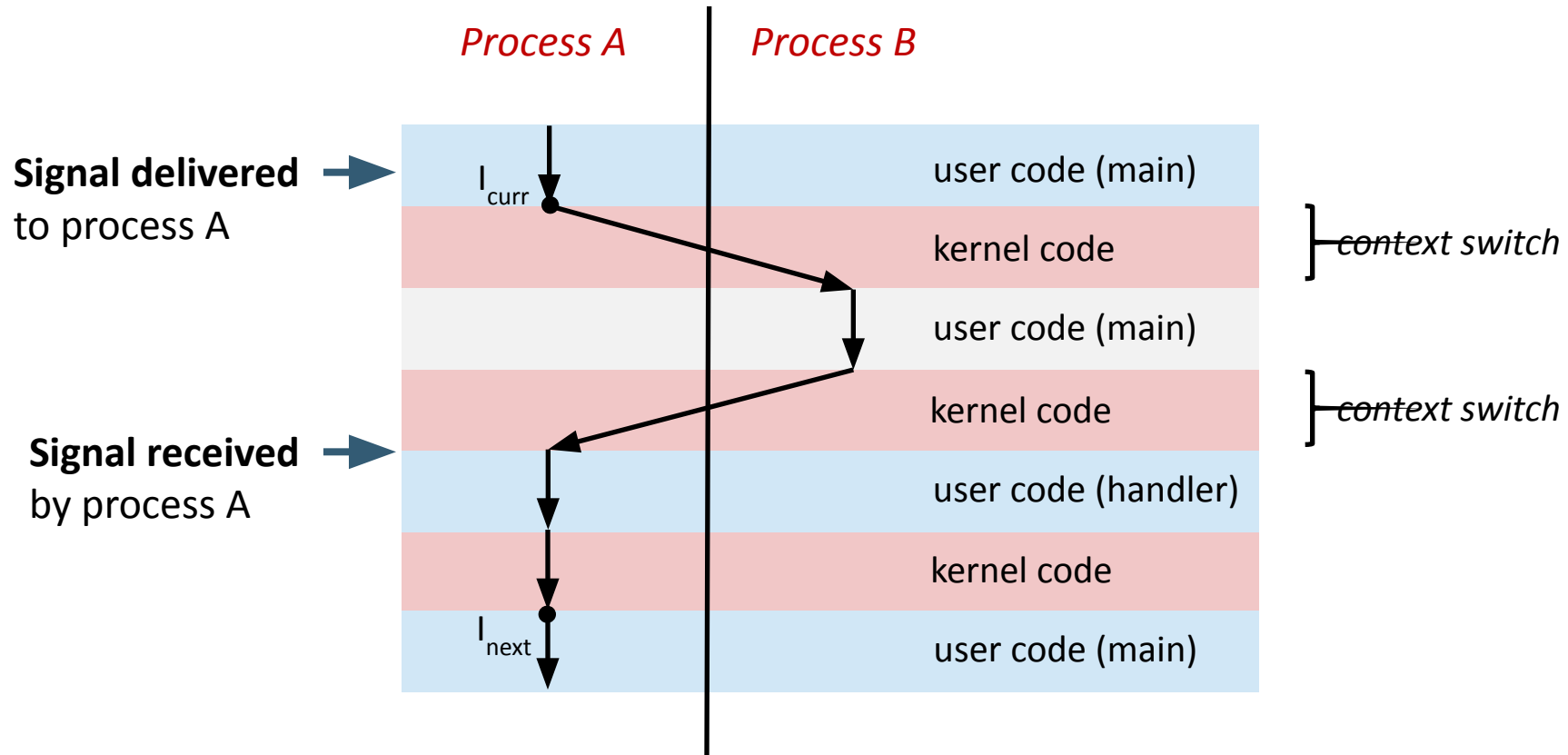
- Suppose kernel is returning from an exception handler and is ready to pass control to process p
- Kernel computes $\text{pnb} = \text{pending} \ \& \ \sim\text{blocked}$
 - The set of pending nonblocked signals for process p
- If $(\text{pnb} == 0)$
 - Pass control to next instruction in the logical flow for p
- Else
 - Choose least nonzero bit k in **pnb** and force process p to *receive* signal k
 - The receipt of the signal triggers some *action* by p
 - Repeat for all nonzero k in **pnb**
 - Pass control to next instruction in logical flow for p

Number	Name	Default action	Corresponding event
1	SIGHUP	Terminate	Terminal line hangup
2	SIGINT	Terminate	Interrupt from keyboard
3	SIGQUIT	Terminate	Quit from keyboard
4	SIGILL	Terminate	Illegal instruction
5	SIGTRAP	Terminate and dump core ^a	Trace trap
6	SIGABRT	Terminate and dump core ^a	Abort signal from abort function
7	SIGBUS	Terminate	Bus error
8	SIGFPE	Terminate and dump core ^a	Floating-point exception
9	SIGKILL	Terminate ^b	Kill program
10	SIGUSR1	Terminate	User-defined signal 1
11	SIGSEGV	Terminate and dump core ^a	Invalid memory reference (seg fault)
12	SIGUSR2	Terminate	User-defined signal 2
13	SIGPIPE	Terminate	Wrote to a pipe with no reader
14	SIGALRM	Terminate	Timer signal from alarm function
15	SIGTERM	Terminate	Software termination signal
16	SIGSTKFLT	Terminate	Stack fault on coprocessor
17	SIGCHLD	Ignore	A child process has stopped or terminated
18	SIGCONT	Ignore	Continue process if stopped
19	SIGSTOP	Stop until next SIGCONT ^b	Stop signal not from terminal
20	SIGTSTP	Stop until next SIGCONT	Stop signal from terminal
21	SIGTTIN	Stop until next SIGCONT	Background process read from terminal
22	SIGTTOU	Stop until next SIGCONT	Background process wrote to terminal
23	SIGURG	Ignore	Urgent condition on socket
24	SIGXCPU	Terminate	CPU time limit exceeded
25	SIGXFSZ	Terminate	File size limit exceeded
26	SIGVTALRM	Terminate	Virtual timer expired
27	SIGPROF	Terminate	Profiling timer expired
28	SIGWINCH	Ignore	Window size changed
29	SIGIO	Terminate	I/O now possible on a descriptor
30	SIGPWR	Terminate	Power failure

DEFAULT ACTIONS

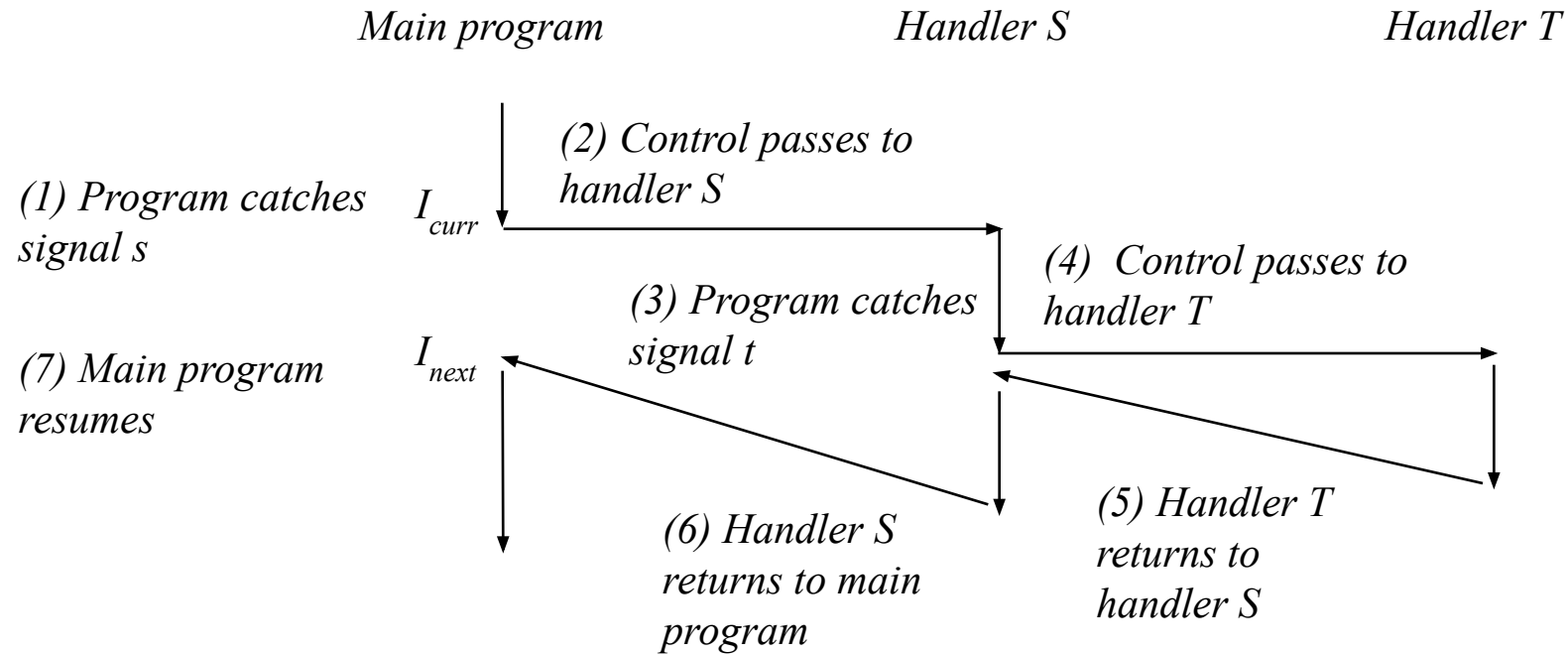
- Each signal type has a predefined *default action*, which is one of:
 - The process terminates
 - The process stops until restarted by a SIGCONT signal
 - The process ignores the signal

VIEW OF SIGNAL HANDLERS AS CONCURRENT FLOWS



NESTED SIGNAL HANDLERS

- Handlers can be interrupted by other handlers



BLOCKING AND UNBLOCKING SIGNALS

- Implicit blocking mechanism
 - Kernel blocks any pending signals of type currently being handled.
 - E.g., A SIGINT handler can't be interrupted by another SIGINT
- Explicit blocking and unblocking mechanism
 - sigprocmask function
- Supporting functions
 - sigemptyset – Create empty set
 - sigfillset – Add every signal number to set
 - sigaddset – Add signal number to set
 - sigdelset – Delete signal number from set

SAFE SIGNAL HANDLING

- Handlers are tricky because they are concurrent with main program and share the same global data structures.- interfere main and other handlers
- How and when signals are received is often counter intuitive
- Different system can have different signal handling semantics

Guidelines

1. Keep handlers as simple as possible
2. Call `async-signal –safe` function in handlers
3. Save and restore `errno`
4. Protect accesses to shared global data structures by blocking all signals
5. Declare global variables with `volatile`
6. Declare flags with `sig_atomic_t`

PORTABLE SIGNAL HANDLING

- Ugh! Different versions of Unix can have different signal handling semantics
 - Some older systems restore action to default after catching signal
 - Some interrupted system calls can return with `errno == EINTR`
 - Some systems don't block signals of the type being handled
- Solution: sigaction

```
handler_t *Signal(int signum, handler_t *handler)
{
    struct sigaction action, old_action;

    action.sa_handler = handler;
    sigemptyset(&action.sa_mask); /* Block sigs of type being
handled */
    action.sa_flags = SA_RESTART; /* Restart syscalls if possible
*/

    if (sigaction(signum, &action, &old_action) < 0)
        unix_error("Signal error");
    return (old_action.sa_handler);
}
```

SYNCHRONIZING FLOWS TO AVOID NASTY CONCURRENCY BUGS

- Parent executes fork function – kernel runs the child
- The child terminates and becomes zombie- delivers SIGCHLD signal to parent
- Before parents gets executed the pending signal is requested by the kernel to be received by parent process
- The signal handler calls deletejob
- After the handler is complete, the kernel runs the parent process and adds child to the job list using addjob

Synchronisation Error- Race

Solution : block SIGCHLD signal

SUMMARY OF SIGNALS

- Signals provide process-level exception handling
 - Can generate from user programs
 - Can define effect by declaring signal handler
 - Be very careful when writing signal handlers
- Nonlocal jumps provide exceptional control flow within process
 - Within constraints of stack discipline